数论板子

这份板子按竞赛中最常见的调用方式整理，所有整数统一使用 `ll = long long`。代码块中的注释只保留传参和返回值，完整说明放在代码块外。

类别	模块
基础	gcd/lcm、exgcd、同余方程、快速幂、逆元、整数开方、整除分块、类欧几里得
素数与分解	线性筛、试除、Miller-Rabin、Pollard-Rho、Min_25
积性函数	phi、mu、莫比乌斯反演、杜教筛、约数个数与约数和
同余	CRT/exCRT、BSGS/exBSGS、原根、Tonelli-Shanks、模高斯消元
组合与代数	组合数、Lucas/扩展 Lucas、二项式反演、Burnside/Polya、斯特林数、分拆、康托、矩阵、NTT
常用序列	Fibonacci 倍增、Catalan、连续异或、常用恒等式

统一约定

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
const ll INF = (1LL << 62);

ll norm(ll x, ll mod) { x %= mod; return x < 0 ? x + mod : x; }
```

`norm` 把整数规范到 `[0, mod)`。涉及负数取模、扩展欧几里得和 CRT 合并时建议统一调用它。

gcd、lcm 与扩展欧几里得

```cpp
// gcd(|a|,|b|)，允许 a、b 为负数
ll gcdll(ll a, ll b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b) { ll t = a % b; a = b; b = t; }
    return a;
}

// lcm(a,b)，结果可能溢出时自行用 i128 检查
ll lcmll(ll a, ll b) { return a / gcdll(a, b) * b; }

// 返回 g=gcd(a,b)，并求 ax+by=g
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = (a >= 0 ? 1 : -1); y = 0; return a >= 0 ? a : -a; }
```

```
    ll x1, y1, g = exgcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1;
    return g;
}

// 解 ax ≡ b (mod m)，返回最小非负解；无解返回 -1
ll linear_mod(ll a, ll b, ll m) {
    a = norm(a, m); b = norm(b, m);
    ll x, y, g = exgcd(a, m, x, y);
    if (b % g) return -1;
    ll mm = m / g;
    return (ll)((i128)x * (b / g) % mm + mm) % mm;
}
```

裴蜀定理：`ax+by=c` 有整数解当且仅当 `gcd(a,b) | c`。`linear_mod` 在模数不必为质数时仍然有效，返回的解模 `m/g` 唯一；所有解为 `x+k(m/g)`。

快速幂、模乘与逆元

```
// a^b mod mod，要求 mod>0，b>=0
ll qpow(ll a, ll b, ll mod) {
    ll r = 1 % mod; a = norm(a, mod);
    while (b) {
        if (b & 1) r = (ll)((i128)r * a % mod);
        a = (ll)((i128)a * a % mod); b >>= 1;
    }
    return r;
}

// 64 位无溢出乘法，0<=a,b<mod，mod<2^63
ll mul_mod(ll a, ll b, ll mod) { return (ll)((i128)a * b % mod); }

// 质数模逆元：mod 为质数且 a%mod != 0
ll inv_prime(ll a, ll mod) { return qpow(a, mod - 2, mod); }

// 任意模逆元：gcd(a,mod)=1，否则返回 -1
ll inv_any(ll a, ll mod) {
    ll x, y, g = exgcd(a, mod, x, y);
    if (g != 1) return -1;
    return norm(x, mod);
}

// 线性求 1..n 在质数 mod 下的逆元，O(n)
vector<ll> inv_all(ll n, ll mod) {
    vector<ll> iv(n + 1); if (n >= 1) iv[1] = 1;
    for (ll i = 2; i <= n; ++i) iv[i] = (mod - mod / i) * iv[mod % i] % mod;
    return iv;
}
```

`__int128` 在 GCC/Clang 下可直接使用；若题目环境没有它，再换浮点模乘。`qpow` 复杂度为 `O(log b)`。

线性筛：质数、最小质因子、欧拉函数、莫比乌斯函数

```
// 传入上界 n；得到 prime、spf、phi、mu（下标 0..n）
struct Sieve {
    int n, pc = 0;
    vector<int> prime, spf, phi, mu;
    Sieve(int n): n(n), spf(n + 1), phi(n + 1), mu(n + 1) {
        mu[1] = 1; phi[1] = 1;
        for (int i = 2; i <= n; ++i) {
            if (!spf[i]) { spf[i] = i; prime.push_back(i); phi[i] = i - 1; mu[i] =
-1; }
            for (int p : prime) {
                if (p > spf[i] || 1LL * p * i > n) break;
                spf[p * i] = p;
                if (i % p == 0) { phi[p * i] = phi[i] * p; mu[p * i] = 0; }
                else { phi[p * i] = phi[i] * (p - 1); mu[p * i] = -mu[i]; }
            }
        }
    }
};
```

线性筛总复杂度 $O(n)$，`spf[x]` 可在 $O(\log x)$ 内分解单个 x。欧拉函数满足 $\varphi(n) = n\prod_{p\mid n}(1-1/p)$；莫比乌斯函数 $\mu(n)$ 在有平方因子时为 0，否则为 $(-1)^{\text{不同质因子个数}}$。

试除分解与约数

```
// 分解 n，返回 {质因子, 指数}；适合 n<=1e12 左右
vector<pair<ll,int>> factor_trial(ll n) {
    vector<pair<ll,int>> r;
    for (ll p = 2; p <= n / p; ++p) if (n % p == 0) {
        int c = 0; while (n % p == 0) n /= p, ++c;
        r.push_back({p, c});
    }
    if (n > 1) r.push_back({n, 1});
    return r;
}

// n 的全部正约数（无序）；传入质因数分解结果
vector<ll> divisors(const vector<pair<ll,int>> &f) {
    vector<ll> d = {1};
    for (auto [p, c] : f) {
        int z = d.size(); ll v = 1;
        for (int e = 1; e <= c; ++e) { v *= p; for (int i = 0; i < z; ++i)
d.push_back(d[i] * v); }
    }
    return d;
}
```

枚举约数只需遍历到 `sqrt(n)`。约数个数为 `Π(e_i+1)`，约数和为 `Π(p_i^{e_i+1}-1)/(p_i-1)`。

64 位 Miller-Rabin 素性测试

```
using u64 = uint64_t;
using u128 = __uint128_t;
// 只要求模数为奇数；支持整个 unsigned 64 位范围。
// 热循环中的乘法没有 128 位除法，比 (u128)a*b%m 快得多。
struct M {
    u64 n, iv, r2, e;

    explicit M(u64 _n) : n(_n) {
        u64 x = n;                          // x = n^(-1) (mod 2^64)
        for (int i = 0; i < 6; ++i) x *= 2 - n * x;
        iv = 0 - x;                         // -n^(-1) (mod 2^64)
        r2 = (u128(0) - n) % n;              // 2^128 mod n
        e = mm(1, r2);                       // Montgomery 表示的 1
    }

    u64 rd(u128 x) const {
        u64 q = (u64)x * iv;
        u128 z = x + (u128)q * n;
        u64 r = (u64)(z >> 64);
        if (z < x) return r - n;             // 处理 128 位加法溢出
        return r >= n ? r - n : r;
    }
    u64 mm(u64 a, u64 b) const { return rd((u128)a * b); }
    u64 in(u64 a) const { return mm(a % n, r2); }
    u64 ad(u64 a, u64 b) const {
        u64 r = a + b;
        return r < a || r >= n ? r - n : r;
    }
    u64 su(u64 a, u64 b) const { return a >= b ? a - b : n - (b - a); }
};

u64 pw(u64 a, u64 b, const M &m) {
    u64 r = m.e, x = m.in(a);
    for (; b; b >>= 1, x = m.mm(x, x))
        if (b & 1) r = m.mm(r, x);
    return r;
}

bool mr(u64 n, const M &m) {                 // 内部接口：n 为奇数且 n >= 3
    u64 d = n - 1, s = __builtin_ctzll(d);
    d >>= s;
    u64 z = m.su(0, m.e);                    // Montgomery 表示的 -1

    // 这组底数对所有 unsigned 64 位整数确定正确。
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL,
                  450775ULL, 9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
```

```
        u64 x = pw(a, d, m);
        if (x == m.e || x == z) continue;
        bool ok = false;
        for (u64 i = 1; i < s; ++i) {
            x = m.mm(x, x);
            if (x == z) { ok = true; break; }
        }
        if (!ok) return false;
    }
    return true;
}

bool is_prime(u64 n) {                          // n: 待判断的 unsigned 64 位整数
    if (n < 2) return false;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                  19ULL, 23ULL, 29ULL, 31ULL, 37ULL})
        if (n % p == 0) return n == p;
    M m(n);
    return mr(n, m);
}
```

`is_prime(n)` 的参数是一个 `u64`，返回 `bool`，对整个 `unsigned 64` 位范围确定正确。例如 `is_prime(1000000007)` 返回 `true`，`is_prime(1000000008)` 返回 `false`。
模数内部必须为奇数；偶数会在函数开头直接判定。编译时请开启 `-O2` 或 `-O3`。

Pollard-Rho 质因数分解（64 位）

```
u64 rg() {
    static u64 s = chrono::steady_clock::now().time_since_epoch().count();
    u64 z = (s += 0x9e3779b97f4a7c15ULL);
    z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
    z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
    return z ^ (z >> 31);
}

u64 gc(u64 a, u64 b) {
    if (!a || !b) return a | b;
    int s = __builtin_ctzll(a | b);
    a >>= __builtin_ctzll(a);
    do {
        b >>= __builtin_ctzll(b);
        if (a > b) swap(a, b);
        b -= a;
    } while (b);
    return a << s;
}

u64 rho(u64 n, const M &m) {                    // n: 合数；返回非平凡因子
    constexpr u64 B = 128, R = 1ULL << 20;
    while (true) {
        u64 y = m.in(rg() % (n - 2) + 2);
```

```
        u64 c = m.in(rg() % (n - 1) + 1);
        auto f = [&](u64 x) { return m.ad(m.mm(x, x), c); };

        u64 x = 0, ys = 0, g = 1, r = 1;
        while (g == 1 && r <= R) {
            x = y;
            for (u64 i = 0; i < r; ++i) y = f(y);
            for (u64 k = 0; k < r && g == 1; k += B) {
                ys = y;
                u64 q = m.e, t = min(B, r - k);
                for (u64 i = 0; i < t; ++i) {
                    y = f(y);
                    q = m.mm(q, m.su(x, y));
                }
                g = gc(q, n);
            }
            r <<= 1;
        }
        if (g == 1) continue;
        if (g == n) {
            do { ys = f(ys); g = gc(m.su(x, ys), n); } while (g == 1);
            if (g == n) continue;
        }
        return g;
    }
}

u64 isqrt(u64 n) {
    u64 r = sqrtl((long double)n);
    while ((u128)r * r > n) --r;
    while ((u128)(r + 1) * (r + 1) <= n) ++r;
    return r;
}

void fd(u64 n, vector<u64> &v) {
    if (n == 1) return;
    u64 r = isqrt(n);
    if ((u128)r * r == n) { fd(r, v); fd(r, v); return; }
    M m(n);
    if (mr(n, m)) { v.push_back(n); return; }
    u64 d = rho(n, m);
    fd(d, v); fd(n / d, v);
}

// 返回 {质因数, 指数}，例如 factor(360) = {{2,3},{3,2},{5,1}}。
vector<pair<u64, int>> factor(u64 n) {
    vector<u64> v;
    for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                  19ULL, 23ULL, 29ULL, 31ULL, 37ULL, 41ULL, 43ULL, 47ULL})
        while (n % p == 0) v.push_back(p), n /= p;
    if (n > 1) fd(n, v);
    sort(v.begin(), v.end());
```

```
    vector<pair<u64, int>> r;
    for (u64 p : v) {
        if (r.empty() || r.back().first != p) r.emplace_back(p, 1);
        else ++r.back().second;
    }
    return r;
}
```

`factor(n)` 的参数是待分解的 `u64`，要求 `n >= 1`，返回 `vector<pair<u64,int>>`：`first` 为质因数，`second` 为其指数，按质因数升序排列。
例如：

```
auto f = factor(360);                       // {{2,3}, {3,2}, {5,1}}
for (auto [p, e] : f) cout << p << '^' << e << '\n';
```

它可以处理 `n < 2^64` 的整数，包括大质数、大质数乘积和重复质因子；`factor(1)` 返回空数组。
Pollard-Rho 只应接收合数，通常不直接调用 `rho`，而是调用 `factor`。
实现使用 Brent 批量 `gcd`，并复用 Montgomery 模乘；相比每轮执行 `(u128)a*b%n` 的写法，64 位大整数分解会明显更快。
若只需判断素性，直接调用上一节的 `is_prime(n)`。

中国剩余定理与扩展 CRT

```
// 互质 CRT：x[i] (mod m[i])，返回 [0,M) 的解；无解返回 -1
ll crt(const vector<ll>& a, const vector<ll>& m) {
    ll x = 0, M = 1;
    for (int i = 0; i < (int)a.size(); ++i) {
        ll t = linear_mod(M, a[i] - x, m[i]);
        if (t < 0) return -1;
        x = (ll)((i128)x + (i128)M * t); M = (ll)((i128)M * m[i]); x = norm(x, M);
    }
    return x;
}

// 非互质 CRT：允许 gcd(mi,mj)>1；返回 x mod lcm，失败返回 -1
ll excrt(const vector<ll>& a, const vector<ll>& m) {
    ll x = norm(a[0], m[0]), M = m[0];
    for (int i = 1; i < (int)a.size(); ++i) {
        ll b = norm(a[i] - x, m[i]), p, q, g = exgcd(M, m[i], p, q);
        if (b % g) return -1;
        ll t = (ll)((i128)p * (b / g) % (m[i] / g));
        ll NM = (ll)((i128)M / g * m[i]);
        x = (ll)(((i128)x + (i128)M * t) % NM); if (x < 0) x += NM; M = NM;
    }
    return x;
}
```

`crt/excrt` 的输入均为同长度数组 `(a[i],m[i])`，表示 `X≡a[i](mod m[i])`。`excrt` 返回最小非负解；若题目要求任意解，直接输出该值即可。

数论分块与整除求和

普通整除分块与 `floor_sum`

```
// 枚举使 floor(n/i) 相同的最大区间 [i,j]，O(sqrt(n))
for (ll i = 1, j; i <= n; i = j + 1) {
    j = n / (n / i);
    // 当前整除块：q=n/i，长度=j-i+1
}

// n：项数；m：分母；a：斜率；b：截距；求 sum_{0<=i<n} floor((a*i+b)/m)
ll floor_sum(ll n, ll m, ll a, ll b) {
    ll ans = 0;
    while (true) {
        if (a >= m) ans += (ll)((i128)(n - 1) * n / 2 * (a / m)), a %= m;
        if (b >= m) ans += n * (b / m), b %= m;
        i128 y = (i128)a * n + b;
        if (y < m) return ans;
        n = (ll)(y / m); b = (ll)(y % m); swap(m, a);
    }
}
```

整除分块常用于 `sum f(floor(n/i))`、`Σ floor(n/i)` 和莫比乌斯/欧拉前缀和，复杂度 $O(\sqrt{n})$。

`floor_sum(n,m,a,b)` 要求 `n>=0`、`m>0`、`a,b>=0`，返回值必须能放入 `ll`；中间乘法已用 `i128`。它和 AtCoder Library 的参数及下标范围一致，复杂度 $O(\log m)$。若 `a` 或 `b` 可能为负，先用带下取整的整除把整倍数提出，不能直接依赖 C++ 向零取整。

高次类欧几里得

下面的完整模块一次求出

$$F_{x,y}(n,a,b,c)=\sum_{i=0}^{n} i^x \left\lfloor \frac{ai+b}{c} \right\rfloor^y \pmod{MOD}, \qquad x+y\le K.$$

适合 `n,a,b,c` 很大，但幂次上界很小的题，例如 `K<=10`。`init()` 只调用一次；随后 `solve(n,a,b,c).f[x][y]` 就是答案。参数要求 `n,a,b>=0`、`c>0`，`MOD` 是大于 `K+1` 的质数。

代码同时维护所有 `x+y<=K` 的状态，递归中才能闭合。若只求普通一次和，前面的 `floor_sum` 更短、更快，不要抄本段。单次复杂度 $O(K^4\log(a+c))$，空间约 $O(K^2\log(a+c))$。

```
const int K = 10;
const ll MOD = 1000000007;

struct Euc {
    ll c[K + 2][K + 2]{}, s[K + 2][K + 2]{}, p[K + 2][K + 2]{};

    struct Ans {
        ll f[K + 1][K + 1];
        Ans() { memset(f, 0, sizeof(f)); }
```

```
    };

    ll pw(ll a, ll b) { // a：底数；b：指数；返回 a^b mod MOD
        ll r = 1;
        for (a %= MOD; b; b >>= 1, a = a * a % MOD)
            if (b & 1) r = r * a % MOD;
        return r;
    }

    void init() { // 预处理组合数和 0^k+...+n^k 的多项式
        c[0][0] = s[0][0] = p[0][0] = 1;
        for (int i = 1; i <= K + 1; i++) {
            c[i][0] = 1;
            for (int j = 1; j <= i; j++) {
                c[i][j] = (c[i - 1][j - 1] + c[i - 1][j]) % MOD;
                s[i][j] = (s[i - 1][j - 1] + j * s[i - 1][j]) % MOD;
            }
            ll d = (2 - i + MOD) % MOD;
            for (int j = 0; j <= i; j++) {
                if (j) p[i][j] = (p[i][j] + p[i - 1][j - 1]) % MOD;
                p[i][j] = (p[i][j] + d * p[i - 1][j]) % MOD;
            }
        }
        ll q[K + 2][K + 2]{};
        for (int k = 0; k <= K; k++)
            for (int j = 0; j <= k; j++) {
                ll w = s[k][j] * pw(j + 1, MOD - 2) % MOD;
                for (int i = 0; i <= j + 1; i++)
                    q[k][i] = (q[k][i] + w * p[j + 1][i]) % MOD;
            }
        memcpy(p, q, sizeof(p));
    }

    ll sum(int k, ll n) { // k：幂次；n：右端点；返回 sum_{i=0}^n i^k
        ll r = 0, v = 1, x = n % MOD;
        for (int i = 0; i <= k + 1; i++) r = (r + p[k][i] * v) % MOD, v = v * x %
MOD;
        return r;
    }

    Ans solve(ll n, ll a, ll b, ll d) {
        // n：右端点；a：斜率；b：截距；d：正分母
        Ans r;
        if (n < 0) return r;
        if (a >= d || b >= d) {
            ll qa = a / d % MOD, qb = b / d % MOD;
            ll pa[K + 1], pb[K + 1]; pa[0] = pb[0] = 1;
            for (int i = 1; i <= K; i++) pa[i] = pa[i - 1] * qa % MOD, pb[i] = pb[i
- 1] * qb % MOD;
            Ans t = solve(n, a % d, b % d, d);
            for (int x = 0; x <= K; x++)
                for (int y = 0; x + y <= K; y++)
```

```
                    for (int j = 0; j <= y; j++)
                        for (int l = 0; l <= y - j; l++) {
                            ll v = c[y][j] * c[y - j][l] % MOD;
                            v = v * pa[l] % MOD * pb[y - j - l] % MOD;
                            r.f[x][y] = (r.f[x][y] + v * t.f[x + l][j]) % MOD;
                        }
            return r;
        }
        ll m = (ll)(((i128)a * n + b) / d);
        if (!m) {
            for (int x = 0; x <= K; x++) r.f[x][0] = sum(x, n);
            return r;
        }
        Ans t = solve(m - 1, d, d - b - 1, a);
        for (int x = 0; x <= K; x++)
            for (int y = 0; x + y <= K; y++) {
                if (!y) { r.f[x][y] = sum(x, n); continue; }
                for (int k = 0; k < y; k++) {
                    ll z = 0;
                    for (int j = 0; j <= x + 1; j++)
                        z = (z + p[x][j] * t.f[k][j]) % MOD;
                    ll v = (sum(k, m - 1) * sum(x, n) - z + MOD) % MOD;
                    r.f[x][y] = (r.f[x][y] + c[y][k] * v) % MOD;
                }
            }
        return r;
    }
};
```

莫比乌斯反演与杜教筛

若 `F(n)=Σ_{d|n} f(d)`，则 `f(n)=Σ_{d|n} μ(d)F(n/d)`。常用恒等式：`Σ_{d|n} μ(d)=[n=1]`，`Σ_{d|n} ϕ(d)=n`。

```
// 预处理 mu 前缀，N<=5e6 时通常足够
struct MuPre {
    int n; vector<int> pr, lp, mu, sm;
    MuPre(int n): n(n), lp(n+1), mu(n+1), sm(n+1) {
        mu[1]=1;
        for(int i=2;i<=n;++i){
            if(!lp[i]) lp[i]=i,pr.push_back(i),mu[i]=-1;
            for(int p:pr){ if(p>lp[i]||1LL*p*i>n)break; lp[p*i]=p; mu[p*i]=(i%p? -
mu[i]:0); }
        }
        for(int i=1;i<=n;++i) sm[i]=sm[i-1]+mu[i];
    }
};
```

杜教筛适合 `n` 很大但只需少量查询，先预处理 `N≈1e6`，再记忆化递归求 `S_mu(n)=Σ_{i≤n}μ(i)` 和 `S_phi(n)`。递归按整除块跳转，单次复杂度约 `O(n^{2/3})`（依预处理大小而定）。

```cpp
// 传入预处理上界 lim；query(n) 返回 sum_{i=1}^n mu(i)
struct DuMu {
    int lim; vector<int> mu, sm; unordered_map<ll,ll> mp;
    DuMu(int lim):lim(lim),mu(lim+1),sm(lim+1){
        vector<int> lp(lim+1),pr; mu[1]=1;
        for(int i=2;i<=lim;++i){if(!lp[i])lp[i]=i,pr.push_back(i),mu[i]=-1;for(int
p:pr){if(p>lp[i]||1LL*p*i>lim)break;lp[p*i]=p;mu[p*i]=(i%p?-mu[i]:0);}}
        for(int i=1;i<=lim;++i)sm[i]=sm[i-1]+mu[i];
    }
    ll query(ll n){
        if(n<=lim)return sm[n]; if(mp.count(n))return mp[n]; ll r=1;
        for(ll l=2,j;l<=n;l=j+1){j=n/(n/l);r-=(j-l+1)*query(n/l);} return mp[n]=r;
    }
};
```

欧拉函数

```cpp
// 单个 phi(n)，试除法 O(sqrt(n))
ll phi(ll n) { ll r=n; for(ll p=2;p<=n/p;++p) if(n%p==0){r=r/p*(p-
1);while(n%p==0)n/=p;} if(n>1)r=r/n*(n-1); return r; }
```

性质：$\gcd(a,b)=1$ 时 $\varphi(ab)=\varphi(a)\varphi(b)$；$\sum_{d\mid n}\varphi(d)=n$；$n>1$ 时简化剩余系之和为 $n\varphi(n)/2$。

离散对数 BSGS / exBSGS

```cpp
// a：底数；b：目标；p：模数；要求 gcd(a,p)=1，无解返回 -1
ll bsgs(ll a,ll b,ll p){
        a=norm(a,p); b=norm(b,p); ll s=sqrt((long double)p)+1;
        unordered_map<ll,ll> tab; tab.reserve((size_t)s*2+1); ll e=1;
        for(ll j=0;j<s;++j){ if(!tab.count(e))tab[e]=j; e=mul_mod(e,a,p); }
        ll as=qpow(a,s,p), iv=inv_any(as,p); if(iv<0)return -1;
        ll cur=b;
        for(ll i=0;i<=s;++i){ if(tab.count(cur)){ll x=i*s+tab[cur];
if(qpow(a,x,p)==b)return x;} cur=mul_mod(cur,iv,p); }
        return -1;
    }
// a：底数；b：目标；m：任意正模数；返回最小非负解，无解返回 -1
ll exbsgs(ll a,ll b,ll m){
        a=norm(a,m); b=norm(b,m); if(m==1||b==1%m)return 0; ll k=1,add=0;
        for(ll g; (g=gcdll(a,m))!=1; ){ if(b%g)return -1;
m/=g;b/=g;k=mul_mod(k,a,m);++add; if(k==b)return add; }
        ll x=bsgs(a,mul_mod(b,inv_any(k,m),m),m); return x<0?-1:x+add;
    }
```

复杂度 `O(sqrt(m))`，哈希表版本常数较小。`exbsgs` 处理底数与模数不互质的情况；如果题目只保证质数模数，调用 `bsgs` 即可。

原根

```
// 求最小原根：n=2,4,p^k,2*p^k（p 为奇质数）时存在，否则返回 -1
ll primitive_root(ll n) {
    if(n==1)return -1; if(n==2)return 1; if(n==4)return 3;
    ll x=n; int c2=0; while(x%2==0) x/=2,++c2;
    // 原根存在当且仅当 n=2,4,p^k,2p^k（p 为奇质数）
    if(c2>=2 || x==1) return -1;
    auto fx=factor_trial(x); if(fx.size()!=1) return -1;
    ll ph=phi(n); auto f=factor_trial(ph);
    for(ll g=2;;++g){if(gcdll(g,n)!=1)continue;bool ok=1;for(auto
[q,e]:f)if(qpow(g,ph/q,n)==1){ok=0;break;}if(ok)return g;}
}
```

判断原根只需对 `ϕ(n)` 的每个不同质因子 `q` 检查 `g^{ϕ(n)/q} != 1 (mod n)`。常见 NTT 模数 `998244353` 的原根为 `3`。

模素数平方根（Tonelli-Shanks）

```
// 求 x^2 ≡ n (mod p) 的最小非负根；p 为奇质数，无解返回 -1
ll mod_sqrt(ll n, ll p) {
    n = norm(n, p); if (!n) return 0; if (p == 2) return n;
    if (qpow(n, (p - 1) / 2, p) != 1) return -1; // Euler 判别
    if (p % 4 == 3) { ll x = qpow(n, (p + 1) / 4, p); return min(x, p - x); }
    ll q = p - 1; int s = 0; while (!(q & 1)) q >>= 1, ++s;
    ll z = 2; while (qpow(z, (p - 1) / 2, p) != p - 1) ++z;
    ll c = qpow(z, q, p), x = qpow(n, (q + 1) / 2, p), t = qpow(n, q, p); int m = s;
    while (t != 1) {
        int i = 1; ll tt = mul_mod(t, t, p);
        while (tt != 1) tt = mul_mod(tt, tt, p), ++i;
        ll b = qpow(c, 1LL << (m - i - 1), p);
        x = mul_mod(x, b, p); c = mul_mod(b, b, p); t = mul_mod(t, c, p); m = i;
    }
    return min(x, p - x);
}
```

Tonelli-Shanks 的复杂度为 `O(log^2 p)`，常用于二次剩余、椭圆曲线和模意义下的几何题。返回另一个根只需 `p-x`。

组合数

质数模、`n < mod`

```cpp
// 初始化到 n；C(n,k) mod mod，mod 为质数且 n<mod
struct Comb {
    ll mod; vector<ll> fac,ifac;
    Comb(int n,ll mod):mod(mod),fac(n+1),ifac(n+1){
        fac[0]=1;for(int i=1;i<=n;++i)fac[i]=fac[i-1]*i%mod;
        ifac[n]=qpow(fac[n],mod-2,mod);for(int i=n;i;--i)ifac[i-1]=ifac[i]*i%mod;
    }
    ll C(int n,int k){if(k<0||k>n)return 0;return fac[n]*ifac[k]%mod*ifac[n-k]%mod;}
};
```

Lucas 定理

`C(n,k) mod p = C(n/p,k/p) * C(n%p,k%p) mod p`。适合 `p` 为质数且 `n` 很大；每层调用小组合数。

```cpp
// p 为质数；预处理 fac/ifac 到 p-1（p 太大时改用乘法计算小组合）
struct Lucas {
    ll p; vector<ll> f,iv;
    Lucas(ll p):p(p),f(p),iv(p){f[0]=1;for(ll i=1;i<p;++i)f[i]=f[i-1]*i%p;iv[p-
1]=qpow(f[p-1],p-2,p);for(ll i=p-1;i;--i)iv[i-1]=iv[i]*i%p;}
    ll small(ll n,ll k){if(k<0||k>n)return 0;return f[n]*iv[k]%p*iv[n-k]%p;}
    ll C(ll n,ll k){if(k<0||k>n)return 0;ll r=1;while(n||k)
{r=r*small(n%p,k%p)%p;n/=p;k/=p;}return r;}
};
```

扩展 Lucas：任意合数模

普通 Lucas 只适用于质数模。要求 $\binom{n}{k} \bmod m$ 且 `m` 是合数时，将 `m` 分解成互质的素数幂，分别求模 p^e 的组合数，再用 CRT 合并。

`ExLucas ex(m)` 的参数 `m` 是固定模数；`ex.C(n,k)` 返回 $\binom{n}{k} \bmod m$。构造会预处理每个素数幂内与对应质数互素的前缀积，适合固定模数、多次询问。`n,k` 可以到 `long long` 范围。

设 `pk` 是 `m` 的某个素数幂因子。预处理和内存是 $O(\sum pk)$，单次询问约为 $O(\sum \log_p n)$。因此应保证每个 `pk` 至多几百万；若模数本身是很大的质数，不应使用本模板。

```cpp
struct ExLucas {
    struct B {
        ll p, pk, e;
        vector<ll> f;
        B(ll p, ll pk, ll e): p(p), pk(pk), e(e), f((size_t)pk + 1) {
            f[0] = 1;
            for (ll i = 1; i <= pk; i++)
                f[i] = i % p ? (ll)((i128)f[i - 1] * i % pk) : f[i - 1];
        }
    };

    ll mod;
    vector<B> v;
```

```cpp
ExLucas(ll m): mod(m) { // m：固定合数模数，要求 m>=1
    ll x = m;
    for (ll p = 2; p <= x / p; p++) if (x % p == 0) {
        ll pk = 1, e = 0;
        while (x % p == 0) x /= p, pk *= p, e++;
        v.emplace_back(p, pk, e);
    }
    if (x > 1) v.emplace_back(x, x, 1);
}

ll pw(ll a, ll b, ll m) { // a：底数；b：指数；m：模数
    ll r = 1 % m;
    for (a %= m; b; b >>= 1, a = (ll)((i128)a * a % m))
        if (b & 1) r = (ll)((i128)r * a % m);
    return r;
}

ll eg(ll a, ll b, ll &x, ll &y) { // 求 ax+by=gcd(a,b)
    if (!b) { x = 1, y = 0; return a; }
    ll u, w, g = eg(b, a % b, u, w);
    x = w, y = u - a / b * w;
    return g;
}

ll iv(ll a, ll m) { // a：被求逆数；m：模数；要求 gcd(a,m)=1
    ll x, y;
    eg(a, m, x, y);
    x %= m;
    return x < 0 ? x + m : x;
}

ll vp(ll n, ll p) { // n：阶乘上界；p：质数；返回 v_p(n!)
    ll r = 0;
    while (n) n /= p, r += n;
    return r;
}

ll fac(ll n, const B &b) { // n：阶乘上界；b：当前素数幂信息
    if (!n) return 1;
    ll r = pw(b.f[b.pk], n / b.pk, b.pk) * b.f[n % b.pk] % b.pk;
    return r * fac(n / b.p, b) % b.pk;
}

ll one(ll n, ll k, const B &b) { // 求 C(n,k) mod b.pk
    if (k < 0 || k > n) return 0;
    ll e = vp(n, b.p) - vp(k, b.p) - vp(n - k, b.p);
    if (e >= b.e) return 0;
    ll r = fac(n, b) * iv(fac(k, b), b.pk) % b.pk;
    r = r * iv(fac(n - k, b), b.pk) % b.pk;
    return r * pw(b.p, e, b.pk) % b.pk;
}
```

```
    ll C(ll n, ll k) { // n：总数；k：选取数；返回 C(n,k) mod mod
        if (k < 0 || k > n || mod == 1) return 0;
        ll r = 0;
        for (const B &b : v) {
            ll a = one(n, k, b), m = mod / b.pk;
            ll z = (ll)((i128)a * m % mod * iv(m % b.pk, b.pk) % mod);
            r = (r + z) % mod;
        }
        return r;
    }
};
```

常用公式：$\binom{n}{k}=\binom{n}{n-k}$，$\binom{n}{k}=\binom{n-1}{k}+\binom{n-1}{k-1}$，$\sum_k\binom{n}{k}=2^n$，$\sum_k\binom{n}{k}^2=\binom{2n}{n}$。范德蒙德卷积为 $\sum_i\binom{a}{i}\binom{b}{k-i}=\binom{a+b}{k}$，Catalan 数为 $\mathrm{Cat}_n=\binom{2n}{n}/(n+1)$。

二项式反演

“至多 `n` 个里恰好选 `i` 个”常对应第一形；“至少 `k` 个里实际有 `i` 个”常对应第二形。反演前先确认求和方向，不要只看组合数长得相似。

第一形：

$$g_n=\sum_{i=0}^{n}\binom{n}{i}f_i\iff f_n=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}g_i.$$

第二形：

$$g_k=\sum_{i=k}^{n}\binom{i}{k}f_i\iff f_k=\sum_{i=k}^{n}(-1)^{i-k}\binom{i}{k}g_i.$$

直接按公式计算是 $O(n^2)$。第一形可写成 `g[n]/n! = sum (f[i]/i!) * (1/(n-i)!)`，因此固定质数模且 `n<mod` 时可用一次卷积做到 $O(n\log n)$；反演只需把第二个序列的第 `j` 项改为 `(-1)^j/j!`。第二形把一侧下标反转后同样是卷积。

Burnside 引理与 Polya 计数

群 `G` 作用在方案集合上时，本质不同方案数为

$$\frac{1}{|G|}\sum_{g\in G}|\operatorname{Fix}(g)|.$$

给环上 `n` 个位置染 `c` 种颜色，旋转 `k` 格会留下 `c^{gcd(n,k)}` 个不变方案。`polya(n,c,flip,mod)` 在 `flip=false` 时只把旋转视为相同，得到项链数；`flip=true` 时连翻转也视为相同，得到手链数。

代码用因数枚举计算全部旋转贡献，复杂度约为分解 `n` 的 $O(\sqrt{n})$ 加因数个数。要求 `mod` 为质数，且群大小在模 `mod` 下可逆：项链要求 `n%mod!=0`，手链要求 `(2n)%mod!=0`。不满足时不能直接“先取模再除”，应使用大整数算精确和后整除，或选择不会整除群大小的多个质数再 CRT。

```
ll polya(ll n, ll c, bool flip, ll mod) {
    // n：环长；c：颜色数；flip：是否视翻转相同；mod：质数
```

```
    assert(n >= 1 && mod >= 2 && n % mod != 0);
    if (flip) assert((__int128_t)2 * n % mod != 0);
    auto mul = [&](ll a, ll b) { return (ll)((__int128_t)a * b % mod); };
    auto pw = [&](ll a, ll b) {
        ll r = 1 % mod;
        for (a %= mod; b; b >>= 1, a = mul(a, a)) if (b & 1) r = mul(r, a);
        return r;
    };

    vector<pair<ll, ll>> ds{{1, 1}}; // {d,phi(d)}，枚举 d|n
    ll x = n;
    for (ll p = 2; p <= x / p; p++) if (x % p == 0) {
        int e = 0;
        while (x % p == 0) x /= p, e++;
        auto old = ds;
        ll q = 1, ph = 1;
        for (int j = 1; j <= e; j++) {
            q *= p, ph = j == 1 ? p - 1 : ph * p;
            for (auto [d, f] : old) ds.push_back({d * q, f * ph});
        }
    }
    if (x > 1) {
        auto old = ds;
        for (auto [d, f] : old) ds.push_back({d * x, f * (x - 1)});
    }

    c = (c % mod + mod) % mod;
    ll sum = 0;
    for (auto [d, ph] : ds) sum = (sum + mul(ph % mod, pw(c, n / d))) % mod;
    if (!flip) return mul(sum, pw(n % mod, mod - 2));

    if (n & 1) sum = (sum + mul(n % mod, pw(c, (n + 1) / 2))) % mod;
    else {
        ll h = (n / 2) % mod;
        sum = (sum + mul(h, pw(c, n / 2 + 1))) % mod;
        sum = (sum + mul(h, pw(c, n / 2))) % mod;
    }
    ll den = mul(2 % mod, n % mod);
    return mul(sum, pw(den, mod - 2));
}
```

精确组合数与任意模数

```
// 精确计算 C(n,k)，结果需能放入 i128；n>=0
i128 C_exact(ll n,ll k){
    if(k<0||k>n)return 0; k=min(k,n-k); i128 r=1;
    for(ll i=1;i<=k;++i)r=r*(n-k+i)/i;
    return r;
}

// 任意模数下计算 C(n,k)；spf[x] 为 1..n 的最小质因子，O(k log n)
```

```
ll C_mod_any(int n,int k,ll mod,const vector<int>& spf){
    if(k<0||k>n)return 0; k=min(k,n-k); unordered_map<int,int> e;
    auto add=[&](int x,int s){while(x>1){int
p=spf[x],c=0;while(x%p==0)x/=p,++c;e[p]+=s*c;}};
    for(int x=n-k+1;x<=n;++x)add(x,1); for(int x=1;x<=k;++x)add(x,-1);
    ll r=1%mod; for(auto [p,c]:e)if(c>0)r=mul_mod(r,qpow(p,c,mod),mod); return r;
}
```

`C_exact` 适合需要真实大整数的题目；`C_mod_any` 不要求模数为质数，因为先在整数层面抵消全部质因子再取模。调用前需用线性筛得到 `spf`，且 `n` 不宜超过预处理上界。

快速斐波那契（倍增）

```
// 返回 (F(n),F(n+1)) mod mod，n>=0
pair<ll,ll> fib(ll n,ll mod){
    if(!n)return {0,1}; auto [a,b]=fib(n>>1,mod);
    ll t=norm((ll)(((i128)2*b-a)%mod),mod);
    ll c=mul_mod(a,t,mod), d=(ll)((i128)mul_mod(a,a,mod)+mul_mod(b,b,mod))%mod;
    return (n&1)?make_pair(d,norm(c+d,mod)):make_pair(c,d);
}
```

复杂度 $O(\log n)$。有 $F_{m+n}=F_mF_{n+1}+F_{m-1}F_n$、$\gcd(F_m,F_n)=F_{\gcd(m,n)}$；当 $n\ge 3$ 时，$F_n\mid F_m$ 当且仅当 $n\mid m$。

连续异或与位运算

```
// 0 xor 1 xor ... xor n，O(1)
ll xor0(ll n){ switch(n&3){case 0:return n;case 1:return 1;case 2:return
n+1;default:return 0;} }
// [l,r] 的异或
ll xor_range(ll l,ll r){ return xor0(r)^xor0(l-1); }
```

恒有 $a+b=(a\oplus b)+2(a\,\&\,b)=(a\mid b)+(a\,\&\,b)$。已知 $X=a+b$、$Y=a\oplus b$，令 $d=(X-Y)/2$；两数解存在当且仅当 $X\ge Y$、$X-Y$ 为偶数且 $(d\,\&\,Y)=0$，此时可取 $(a,b)=(d,d+Y)$。

NTT（998244353）

```
const ll MOD998=998244353, G998=3;
void ntt998(vector<ll>& a,bool iv){ // a：系数数组；iv：是否逆变换
        int n=a.size(); for(ll&x:a)x=norm(x,MOD998); for(int i=1,j=0;i<n;++i){int
b=n>>1;for(;j&b;b>>=1)j^=b;j^=b;if(i<j)swap(a[i],a[j]);}
        for(int len=2;len<=n;len<<=1){ll wlen=qpow(G998,(MOD998-
1)/len,MOD998);if(iv)wlen=qpow(wlen,MOD998-2,MOD998);for(int i=0;i<n;i+=len){ll
w=1;for(int j=0;j<len/2;++j){ll u=a[i+j],v=a[i+j+len/2]*w%MOD998;a[i+j]=
(u+v)%MOD998;a[i+j+len/2]=(u-v+MOD998)%MOD998;w=w*wlen%MOD998;}}}
        if(iv){ll ni=qpow(n,MOD998-2,MOD998);for(ll&x:a)x=x*ni%MOD998;}
    }
vector<ll> mul998(vector<ll>a,vector<ll>b){ // a、b：多项式系数；返回模 998244353 卷积
    if(a.empty()||b.empty())return {};
    int
z=a.size()+b.size()-1,n=1;while(n<z)n<<=1;a.resize(n);b.resize(n);ntt998(a,0);ntt998
(b,0);for(int i=0;i<n;++i)a[i]=a[i]*b[i]%MOD998;ntt998(a,1);a.resize(z);return a;
}
```

卷积复杂度 `O(n log n)`，长度需满足 `n` 为 `2` 的幂且不超过 `2^23`。若使用其他 NTT 模数，需要替换 `mod` 和原根。

整数开方与有符号整除

```
// floor(sqrt(n))，n>=0，避免 sqrtl 的舍入误差
ll isqrt(ll n) {
    ll x = sqrtl((long double)n);
    while ((x + 1) > 0 && (x + 1) <= n / (x + 1)) ++x;
    while (x > n / x) --x;
    return x;
}

// floor(a/b)、ceil(a/b)，要求 b>0；适用于 a 为负数
ll floor_div(ll a, ll b) { return a >= 0 ? a / b : -((-a + b - 1) / b); }
ll ceil_div(ll a, ll b) { return a >= 0 ? (a + b - 1) / b : -((-a) / b); }
```

`isqrt` 常用于二次筛、判完全平方数和 Pollard-Rho 前置判断。注意 `LLONG_MIN` 的绝对值无法用 `ll` 表示，若题目允许该边界需改用 `i128`。

Min_25 筛：质数和

下面的版本求 `Σ_{p≤n} p (mod mod)`，其中 `n` 可到 `1e11` 左右，`mod` 取奇数模数。它使用整除分块保存 `n/i` 的所有不同取值，空间和时间约为 `O(n^{3/4}/log n)`。

```
// 返回 sum_{p<=n} p mod mod；要求 n>=1，mod 为奇数且中间答案按模计算
struct Min25Prime {
    ll n, mod, rt, iv2; vector<ll> w, g, ps; vector<int> id1, id2, pr;
    Min25Prime(ll n, ll mod): n(n), mod(mod) {
        rt = sqrtl((long double)n);
        while ((rt + 1) <= n / (rt + 1)) ++rt;
        while (rt > n / rt) --rt;
```

```
        id1.assign(rt + 1, -1); id2.assign(rt + 1, -1); iv2 = (mod + 1) / 2;
        for (ll l = 1, r; l <= n; l = r + 1) {
            r = n / (n / l); ll x = n / l; int i = w.size();
            w.push_back(x); g.push_back((ll)((i128)(x % mod) * ((x + 1) % mod) % mod
* iv2 - 1));
            g.back() = norm(g.back(), mod);
            if (x <= rt) id1[x] = i; else id2[n / x] = i;
        }
        vector<bool> vis(rt + 1);
        for (int i = 2; i <= rt; ++i) {
            if (!vis[i]) pr.push_back(i);
            for (int p : pr) { if (1LL * p * i > rt) break; vis[p * i] = true; if (i
% p == 0) break; }
        }
        ps.assign(pr.size() + 1, 0);
        for (int i = 0; i < (int)pr.size(); ++i) ps[i + 1] = (ps[i] + pr[i]) % mod;
        for (int i = 0; i < (int)pr.size(); ++i) {
            ll p = pr[i];
            for (int j = 0; j < (int)w.size() && p * p <= w[j]; ++j) {
                ll v = g[id(w[j] / p)];
                g[j] = norm(g[j] - (ll)((i128)p * norm(v - ps[i], mod) % mod), mod);
            }
        }
    }
    int id(ll x) const { return x <= rt ? id1[x] : id2[n / x]; }
    ll solve() const { return g[id(n)]; }
};
```

初始值 `x(x+1)/2-1` 是 `2..x` 的整数和；每处理质数 `p`，剔除含有 `p` 的合数贡献。若要计算其他积性函数，需要把初始 `g` 和转移式替换为相应的 Min_25 递推。

杜教筛同时求 `sum mu` 与 `sum phi`

```
// 传入预处理上界 lim；mu_sum(n)、phi_sum(n) 返回前缀和，结果需能放入 ll
struct DuSieve {
    int lim; vector<int> lp, pr, mu, ph; vector<ll> smu, sphi;
    unordered_map<ll,ll> mm, mp;
    DuSieve(int lim):lim(lim),lp(lim+1),mu(lim+1),ph(lim+1),smu(lim+1),sphi(lim+1) {
        mu[1]=1; ph[1]=1;
        for(int i=2;i<=lim;++i){
            if(!lp[i]) lp[i]=i,pr.push_back(i),mu[i]=-1,ph[i]=i-1;
            for(int p:pr){
                if(p>lp[i]||1LL*p*i>lim)break; lp[p*i]=p;
                if(i%p==0) mu[p*i]=0,ph[p*i]=ph[i]*p;
                else mu[p*i]=-mu[i],ph[p*i]=ph[i]*(p-1);
            }
        }
        for(int i=1;i<=lim;++i)smu[i]=smu[i-1]+mu[i],sphi[i]=sphi[i-1]+ph[i];
    }
    ll mu_sum(ll n){
        if(n<=lim)return smu[n]; if(mm.count(n))return mm[n]; ll r=1;
```

```
        for(ll l=2,j;l<=n;l=j+1){j=n/(n/l);r-=(j-l+1)*mu_sum(n/l);} return mm[n]=r;
    }
    ll phi_sum(ll n){
        if(n<=lim)return sphi[n]; if(mp.count(n))return mp[n];
        ll r=(ll)((i128)n*(n+1)/2);
        for(ll l=2,j;l<=n;l=j+1){j=n/(n/l);r-=(j-l+1)*phi_sum(n/l);} return mp[n]=r;
    }
};
```

`mu_sum` 的递推来自 $\sum_{d=1}^{n}\sum_{k\mid d}\mu(k)=1\ (n\ge 1)$；`phi_sum` 来自 $\sum_{d\mid n}\varphi(d)=n$。预处理上界通常取 `1e6`，查询量多时应增大上界以减少哈希递归。

斯特林数与整数分拆

```
// 第二类斯特林数 S(n,k)：返回二维表，O(nk)，模 mod
vector<vector<ll>> stirling2(int n, int k, ll mod) {
    vector<vector<ll>> s(n + 1, vector<ll>(k + 1)); s[0][0] = 1;
    for(int i=1;i<=n;++i) for(int j=1;j<=min(i,k);++j)
        s[i][j] = ((i128)j * s[i-1][j] + s[i-1][j-1]) % mod;
    return s;
}

// 第一类无符号斯特林数 c(n,k)：c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k)
vector<vector<ll>> stirling1(int n, int k, ll mod) {
    vector<vector<ll>> s(n + 1, vector<ll>(k + 1)); s[0][0] = 1;
    for(int i=1;i<=n;++i) for(int j=1;j<=min(i,k);++j)
        s[i][j] = (s[i-1][j-1] + (i128)(i-1) * s[i-1][j]) % mod;
    return s;
}

// 整数分拆数 p(0..n)，p[t] 表示 t 的无序正整数分拆数，O(n^2)
vector<ll> partition_num(int n, ll mod) {
    vector<ll> dp(n + 1); dp[0] = 1;
    for(int x=1;x<=n;++x) for(int s=x;s<=n;++s) dp[s]=(dp[s]+dp[s-x])%mod;
    return dp;
}
```

球不同盒不同且允许空盒为 `m^n`；球不同盒同且允许空盒为 `Σ_{k=0}^m S(n,k)`；球同盒不同且允许空盒为 `C(n+m-1,m-1)`。分拆数生成函数为 `Π_{i≥1}(1-x^i)^{-1}`。

矩阵快速幂与线性递推

```
// 固定维度矩阵，矩阵下标 0..DIM-1
template<int DIM>
struct Mat {
    ll a[DIM][DIM]{};
};

// x*y mod mod，O(DIM^3)
template<int DIM>
```

```
Mat<DIM> mat_mul(const Mat<DIM>& x, const Mat<DIM>& y, ll mod) {
    Mat<DIM> r;
    for(int i=0;i<DIM;++i)for(int k=0;k<DIM;++k)if(x.a[i][k])for(int j=0;j<DIM;++j)
        r.a[i][j]=(r.a[i][j]+(i128)x.a[i][k]*y.a[k][j])%mod;
    return r;
}

// A^e mod mod，O(DIM^3 log e)
template<int DIM>
Mat<DIM> mat_pow(Mat<DIM> a, ll e, ll mod) {
    Mat<DIM> r; for(int i=0;i<DIM;++i)r.a[i][i]=1%mod;
    while(e){if(e&1)r=mat_mul(r,a,mod);a=mat_mul(a,a,mod);e>>=1;} return r;
}
```

矩阵快速幂复杂度为 `O(DIM^3 log e)`；线性递推的初始项和转移式可用 `DIM` 阶矩阵直接加速。维度较大且矩阵稀疏时，保留代码中的 `if(x.a[i][k])` 可明显减小常数。

p-进制阶乘指数与组合数整除性

```
// v_p(n!)：n! 中质因子 p 的指数，p 为质数
ll vp_fact(ll n, ll p) { ll r=0; for(;n;n/=p) r+=n/p; return r; }

// v_p(C(n,k))：可用于判断组合数是否被 p 整除
ll vp_comb(ll n,ll k,ll p){return vp_fact(n,p)-vp_fact(k,p)-vp_fact(n-k,p);}
```

`v_p(n!)=⌊n/p⌋+⌊n/p^2⌋+...`（Legendre 公式）。`v_p(C(n,k))` 等于 `n` 减去 `k` 与 `n-k` 的二进制/`p` 进制进位次数（Kummer 定理）。例如 `C(n,k)` 的末尾零个数为 `min(v_2,v_5)`。

模意义高斯消元

```
// 增广矩阵 a[n][m+1]，mod 为质数；返回 0 唯一解、1 多解、2 无解
int gauss_mod(vector<vector<ll>>& a,ll mod){
    int n=a.size(),m=a[0].size()-1,r=0;
    for(int c=0;c<m&&r<n;++c){
        int t=r; for(int i=r+1;i<n;++i)if(a[i][c]>a[t][c])t=i;
        if(!a[t][c])continue; swap(a[t],a[r]);
        ll iv=qpow(a[r][c],mod-2,mod);
        for(int j=c;j<=m;++j)a[r][j]=a[r][j]*iv%mod;
        for(int i=0;i<n;++i)if(i!=r&&a[i][c]){
            ll z=a[i][c]; for(int j=c;j<=m;++j)a[i][j]=norm(a[i][j]-(ll)
((i128)z*a[r][j]%mod),mod);
        }
        ++r;
    }
    for(int i=r;i<n;++i){bool all0=1;for(int j=0;j<m;++j)if(a[i]
[j])all0=0;if(all0&&a[i][m])return 2;}
    return r==m?0:1;
}
```

矩阵为唯一解时，第 `j` 个未知数就在主元行的最后一列；多解时自由元可自行赋值。若模数不是质数，把主元逆元改为 `inv_any`，并先确认主元与模数互质。

康托展开

```
// 全排列 p[0..n-1]（元素为 1..n），返回 0-based 字典序排名 mod mod
ll cantor_rank(const vector<int>& p,ll mod){
    int n=p.size(); vector<ll> fac(n+1); fac[0]=1;
    for(int i=1;i<=n;++i)fac[i]=fac[i-1]*i%mod;
    vector<int> bit(n+1); auto add=[&](int x){for(;x<=n;x+=x&-x)++bit[x];};
    auto sum=[&](int x){int r=0;for(;x;x-=x&-x)r+=bit[x];return r;};
    for(int i=1;i<=n;++i)add(i); ll ans=0;
    for(int i=0;i<n;++i){ll less=sum(p[i]-1);ans=(ll)((ans+(i128)less*fac[n-1-
i])%mod);for(int x=p[i];x<=n;x+=x&-x)--bit[x];}
    return ans;
}
```

不取模时把 `fac` 和 `ans` 换成足够大的整数。若题目把最小排列记作第 1 个，最后输出 `(rank+1)%mod`。

常见结论速查

整除、最大公约数与最小公倍数

- $a \mid b$ 表示存在整数 k 使 $b = ka$。若 $a \mid b$ 且 $a \mid c$，则 $a \mid (xb + yc)$ 对任意整数 x, y 成立。
- $\gcd(a, b) = \gcd(b, a \bmod b) = \gcd(a, b - a) = \gcd(a, b + ka)$。
- $\gcd(ka, kb) = |k| \gcd(a, b)$，$\operatorname{lcm}(ka, kb) = |k| \operatorname{lcm}(a, b)$。
- $\gcd(a, b) \operatorname{lcm}(a, b) = |ab|$。
- 若 $\gcd(a, b) = 1$ 且 $a \mid n, b \mid n$，则 $ab \mid n$；若 $a \mid bc$ 且 $\gcd(a, b) = 1$，则 $a \mid c$。
- $\gcd(a_1, a_2, \ldots, a_n) = \gcd(a_1, a_2 - a_1, \ldots, a_n - a_{n-1})$，含公共加数的 gcd 问题常可转为差分。
- 令 $g = \gcd(a, m)$。条件 $\gcd(a + x, m) = g$ 等价于 $g \mid x$ 且

$$\gcd\left(\frac{a}{g} + \frac{x}{g}, \frac{m}{g}\right) = 1.$$

因此按 $x \bmod m$ 计，解的数量是

$$\varphi\left(\frac{m}{\gcd(a, m)}\right).$$

- 线性丢番图方程 $ax + by = c$ 有整数解当且仅当 $\gcd(a, b) \mid c$。若 (x_0, y_0) 是一组解且 $g = \gcd(a, b)$，则全部解为

$$x = x_0 + k\frac{b}{g}, \qquad y = y_0 - k\frac{a}{g}, \qquad k \in \mathbb{Z}.$$

质因数分解与约数函数

设

$$n = \prod_{i=1}^{r} p_i^{e_i}.$$

- 正约数个数：$\tau(n) = \prod_{i=1}^{r} (e_i + 1)$。
- 正约数和：$\sigma(n) = \prod_{i=1}^{r} \dfrac{p_i^{e_i+1} - 1}{p_i - 1}$。
- 全部正约数之积：$\prod_{d \mid n} d = n^{\tau(n)/2}$。
- a, b 的公约数个数为 $\tau(\gcd(a,b))$，公约数之和为 $\sigma(\gcd(a,b))$。
- n 为完全平方数当且仅当 $\tau(n)$ 为奇数。
- k 次幂自由（不存在 $p^k \mid n$）可通过质因子指数均小于 k 判断；平方自由等价于 $\mu(n) \neq 0$。
- $n!$ 中质因子 p 的指数为

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor.$$

- 组合数中质因子 p 的指数为

$$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p((n-k)!).$$

幂、估值与常用整除结论

- 对整数 $a > 1$ 和正整数 m, n，有

$$\gcd(a^m - 1, a^n - 1) = a^{\gcd(m,n)} - 1.$$

因此 $a^m - 1 \mid a^n - 1$ 当且仅当 $m \mid n$。

- 几何级数恒等式：

$$1 + a + a^2 + \cdots + a^{n-1} = \frac{a^n - 1}{a - 1}.$$

在模意义下若 $a - 1$ 不可逆，不能直接除，应用分治或矩阵计算几何和。

- LTE（奇质数版本）：若奇质数 $p \mid x - y$ 且 $p \nmid xy$，则

$$v_p(x^n - y^n) = v_p(x - y) + v_p(n).$$

- 若奇质数 $p \mid x + y$、$p \nmid xy$ 且 n 为奇数，则

$$v_p(x^n + y^n) = v_p(x + y) + v_p(n).$$

- LTE 的 $p = 2$ 常用形式：若 x, y 均为奇数且 n 为偶数，则

$$v_2(x^n - y^n) = v_2(x - y) + v_2(x + y) + v_2(n) - 1.$$

若 n 为奇数，则 $v_2(x^n - y^n) = v_2(x - y)$。

- 若质数 $p \mid ab$，则 $p \mid a$ 或 $p \mid b$；这是质数区别于一般合数的 Euclid 引理。
- 质数计数函数满足素数定理

$$\pi(x) \sim \frac{x}{\ln x},$$

因此第 n 个质数的量级约为 $n \ln n$，只应用于估算筛法空间，不是精确上界。

欧拉函数

- 定义：$\varphi(n)$ 是 $1 \le x \le n$ 中满足 $\gcd(x,n)=1$ 的整数个数，且 $\varphi(1)=1$。
- 质数幂：$\varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p-1)$。
- 一般公式：

$$\varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right).$$

- 若 $\gcd(m,n)=1$，则 $\varphi(mn)=\varphi(m)\varphi(n)$。
- 对正整数 k，有 $\varphi(n^k) = n^{k-1}\varphi(n)$。
- 若 p 为质数，则

$$\varphi(pn) = \begin{cases} p\varphi(n), & p \mid n, \\ (p-1)\varphi(n), & p \nmid n. \end{cases}$$

- 特别地，n 为奇数时 $\varphi(2n)=\varphi(n)$，n 为偶数时 $\varphi(2n)=2\varphi(n)$。
- 不要求 m,n 互质时，有重要恒等式

$$\boxed{\varphi(mn)\varphi(\gcd(m,n)) = \varphi(m)\varphi(n)\gcd(m,n)}.$$

- 约数和恒等式：

$$\sum_{d \mid n} \varphi(d) = n.$$

- 对 $n>1$，所有小于等于 n 且与 n 互质的数之和为

$$\sum_{\substack{1 \le k \le n \\ \gcd(k,n)=1}} k = \frac{n\varphi(n)}{2}.$$

- 当 $n>2$ 时 $\varphi(n)$ 为偶数。
- 若 $d \mid n$，则 $1 \le x \le n$ 中满足 $\gcd(x,n)=d$ 的 x 有 $\varphi(n/d)$ 个。
- $1 \le a,b \le n$ 中满足 $\gcd(a,b)=1$ 的有序对数量为

$$2\sum_{k=1}^{n} \varphi(k) - 1.$$

- 阶数为 n 的 Farey 序列长度为

$$1 + \sum_{k=1}^{n} \varphi(k).$$

- gcd 求和：

$$\sum_{i=1}^{n}\gcd(i,n)=\sum_{d\mid n}d\,\varphi\Big(\frac{n}{d}\Big)=\sum_{d\mid n}\frac{n}{d}\varphi(d).$$

莫比乌斯函数、反演与狄利克雷卷积

- 莫比乌斯函数定义为

$$\mu(n)=\begin{cases}1, & n=1,\\ (-1)^r, & n\text{ 是 }r\text{ 个不同质数之积},\\ 0, & n\text{ 含平方因子}.\end{cases}$$

- 基本恒等式：

$$\sum_{d\mid n}\mu(d)=[n=1],\qquad \sum_{d\mid n}\frac{\mu(d)}{d}=\frac{\varphi(n)}{n},\qquad \varphi(n)=\sum_{d\mid n}\mu(d)\frac{n}{d}.$$

- 若 $F(n)=\sum_{d\mid n}f(d)$，则莫比乌斯反演为

$$f(n)=\sum_{d\mid n}\mu(d)F\Big(\frac{n}{d}\Big).$$

- 若 $F(n)=\sum_{n\mid d}f(d)$，则其“倍数形式”反演为

$$f(n)=\sum_{n\mid d}\mu\left(\frac{d}{n}\right)F(d).$$

- 狄利克雷卷积定义为 $(f*g)(n)=\sum_{d\mid n}f(d)g(n/d)$。记恒等函数 $\mathrm{id}(n)=n$，则

$$\mathbf{1}*\mu=\varepsilon,\qquad \varphi=\mathrm{id}*\mu,\qquad \mathbf{1}*\varphi=\mathrm{id}.$$

- φ,μ,τ,σ 都是积性函数：若 $\gcd(m,n)=1$，则 $f(mn)=f(m)f(n)$。因此只需知道 $f(p^k)$ 就能由质因数分解计算 $f(n)$。

同余、逆元、阶与中国剩余定理

- $a\equiv b\pmod m$ 当且仅当 $m\mid(a-b)$；同余可以相加、相减、相乘，但除法必须乘逆元。
- a 模 m 的逆元存在当且仅当 $\gcd(a,m)=1$。
- 线性同余 $ax\equiv b\pmod m$ 有解当且仅当 $g=\gcd(a,m)$ 整除 b；有解时模 m 恰有 g 个不同解。
- 欧拉定理：若 $\gcd(a,m)=1$，则

$$a^{\varphi(m)}\equiv 1\pmod m.$$

- 费马小定理：质数 p 满足 $a^p\equiv a\pmod p$；若 $p\nmid a$，则 $a^{p-1}\equiv 1\pmod p$。
- Wilson 定理：p 为质数当且仅当

$$(p-1)!\equiv -1\pmod p.$$

- Carmichael 函数满足 $a^{\lambda(n)}\equiv 1\pmod n$ $(\gcd(a,n)=1)$ 。对奇质数幂 $\lambda(p^k)=\varphi(p^k)$；$k\ge 3$ 时 $\lambda(2^k)=2^{k-2}$；互质质数幂之间取 lcm。

- 具体地，$\lambda(2)=1$、$\lambda(4)=2$，且对互质 m,n 有 $\lambda(mn)=\operatorname{lcm}(\lambda(m),\lambda(n))$。
- 模 m 下 a 的乘法阶 $\operatorname{ord}_m(a)$ 一定整除 $\varphi(m)$，并且

$$\operatorname{ord}_m(a^k)=\frac{\operatorname{ord}_m(a)}{\gcd(\operatorname{ord}_m(a),k)}.$$

- 若模数存在原根，则原根个数为 $\varphi(\varphi(m))$。原根存在当且仅当

$$m\in\{2,4,p^k,2p^k\},$$

其中 p 为奇质数。

- 质数 p 下方程 $x^k\equiv 1\pmod p$ 恰有 $\gcd(k,p-1)$ 个非零解。
- 一般 CRT 方程组 $x\equiv a_i\pmod{m_i}$ 有解当且仅当任意两式满足

$$a_i\equiv a_j\pmod{\gcd(m_i,m_j)}.$$

有解时答案模 $\operatorname{lcm}(m_1,\dots,m_k)$ 唯一。

二次剩余与平方和

- Euler 判别：奇质数 p 且 $p\nmid a$ 时

$$a^{(p-1)/2}\equiv\begin{cases}1\pmod p, & a\text{ 是二次剩余},\\ -1\pmod p, & a\text{ 是二次非剩余}.\end{cases}$$

- Legendre 符号满足乘法性

$$\left(\frac{ab}{p}\right)=\left(\frac{a}{p}\right)\left(\frac{b}{p}\right).$$

- 两个常用补充律：

$$\left(\frac{-1}{p}\right)=(-1)^{(p-1)/2},\qquad \left(\frac{2}{p}\right)=(-1)^{(p^2-1)/8}.$$

- 二次互反律（不同奇质数 p,q）：

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right)=(-1)^{\frac{p-1}{2}\frac{q-1}{2}}.$$

- Fermat 两平方和定理：正整数 n 能表示成两个整数平方之和，当且仅当所有满足 $p\equiv 3\pmod 4$ 的质因子在 n 中指数均为偶数。
- 本原勾股数可唯一写成（交换两直角边除外）

$$(a,b,c)=(u^2-v^2,\ 2uv,\ u^2+v^2),$$

其中 $u>v>0$、$\gcd(u,v)=1$ 且 u,v 奇偶不同。

取整、取模与调和级数

- $a \bmod m = a - m\lfloor a/m \rfloor$；需要非负余数时统一规范到 $[0, m)$。
- 嵌套取整：

$$\left\lfloor \frac{\lfloor n/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor.$$

- $\lfloor n/i \rfloor = q$ 对应的完整下标区间为

$$\left[\left\lfloor \frac{n}{q+1} \right\rfloor + 1, \left\lfloor \frac{n}{q} \right\rfloor \right].$$

- $\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 个不同取值；当前左端点为 l 时，右端点为 $r = \lfloor n/\lfloor n/l \rfloor \rfloor$。
- 约数计数前缀满足

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = \sum_{k=1}^{n} \tau(k).$$

- 调和数近似为

$$H_n = \sum_{k=1}^{n} \frac{1}{k} = \ln n + \gamma + \frac{1}{2n} + O\left(\frac{1}{n^2}\right), \qquad \gamma \approx 0.5772156649.$$

常用求和与组合恒等式

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \qquad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}, \qquad \sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2.$$

- 对称性与 Pascal 恒等式：

$$\binom{n}{k} = \binom{n}{n-k}, \qquad \binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}.$$

- 常用变形：

$$k\binom{n}{k} = n\binom{n-1}{k-1}, \qquad \binom{n}{k}\binom{k}{m} = \binom{n}{m}\binom{n-m}{k-m}.$$

- 二项式求和：

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n} (-1)^k \binom{n}{k} = [n=0].$$

$$\sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}, \qquad \sum_{k=0}^{n} k^2\binom{n}{k} = n(n+1)2^{n-2}.$$

- 平方和与范德蒙德卷积：

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}, \qquad \sum_{i=0}^{k} \binom{a}{i}\binom{b}{k-i} = \binom{a+b}{k}.$$

- 曲棍球棒恒等式：

$$\sum_{i=r}^{n}\binom{i}{r}=\binom{n+1}{r+1}.$$

- 二项式反演：

$$f_n=\sum_{i=0}^{n}\binom{n}{i}g_i\iff g_n=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f_i.$$

- Lucas 定理（p 为质数）：把 n,k 写成 p 进制 $n=\sum n_ip^i,k=\sum k_ip^i$，则

$$\binom{n}{k}\equiv\prod_i\binom{n_i}{k_i}\pmod p.$$

- Kummer 定理：$v_p\binom{n}{k}$ 等于 p 进制下计算 $k+(n-k)$ 时的进位次数。特别地，

$$\binom{n}{k}\equiv 1\pmod 2\iff (k\,\&\,(n-k))=0\iff (k\,\&\sim n)=0.$$

- 错排数：

$$D_n=(n-1)(D_{n-1}+D_{n-2})=n!\sum_{k=0}^{n}\frac{(-1)^k}{k!},\qquad D_0=1,D_1=0.$$

- 非负整数解 $x_1+\cdots+x_n=m$ 的数量为 $\binom{m+n-1}{n-1}$；若再要求 $0\le x_i<k$，方案数为

$$\sum_{i=0}^{\lfloor m/k\rfloor}(-1)^i\binom{n}{i}\binom{m-ik+n-1}{n-1},$$

其中组合数上标小于下标时按 0 处理。

Catalan、Stirling、分拆与球盒模型

- Catalan 数：

$$\mathrm{Cat}_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n-1}.$$

$$\mathrm{Cat}_0=1,\qquad \mathrm{Cat}_n=\sum_{i=0}^{n-1}\mathrm{Cat}_i\,\mathrm{Cat}_{n-1-i},\qquad \mathrm{Cat}_n=\mathrm{Cat}_{n-1}\frac{4n-2}{n+1}.$$

- Catalan 模型包括：n 对合法括号、n 个元素合法出栈序列、n 个结点的二叉树、从 $(0,0)$ 到 (n,n) 且不越过对角线的单调路径。
- 第二类 Stirling 数：

$$S(n,k)=kS(n-1,k)+S(n-1,k-1),$$

表示把 n 个不同元素划分为 k 个非空无标号集合的方案数。

- 第一类无符号 Stirling 数：

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),$$

表示 n 个元素恰好形成 k 个置换环的排列数。

- Bell 数为 $B_n = \sum_{k=0}^{n} S(n,k)$。
- 整数分拆生成函数：

$$\sum_{n\geq 0} p(n)x^n = \prod_{i\geq 1} \frac{1}{1-x^i}.$$

- 球同、盒不同、盒可空：$\binom{n+m-1}{m-1}$。
- 球同、盒不同、盒非空：$\binom{n-1}{m-1}$。
- 球不同、盒不同、盒可空：m^n。
- 球不同、盒不同、盒非空：$m!S(n,m)$。
- 球不同、盒相同、盒非空：$S(n,m)$。
- 球不同、盒相同、最多使用 m 个盒：$\sum_{k=0}^{m} S(n,k)$。
- 球同、盒相同的两种情况分别等价于把 n 分拆成恰好 m 个正整数、或至多 m 个正整数。

斐波那契数列

取 $F_0 = 0, F_1 = 1$。

- 加法公式：

$$F_{m+n} = F_m F_{n+1} + F_{m-1} F_n.$$

- 倍增公式：

$$F_{2n} = F_n(2F_{n+1} - F_n), \qquad F_{2n+1} = F_n^2 + F_{n+1}^2.$$

- Cassini 恒等式：

$$F_{n-1}F_{n+1} - F_n^2 = (-1)^n.$$

- gcd 性质：

$$\gcd(F_m, F_n) = F_{\gcd(m,n)}.$$

- 当 $m \geq 3$ 时，$F_m \mid F_n$ 当且仅当 $m \mid n$；$F_1 = F_2 = 1$ 需单独处理。
- 前缀和与平方和：

$$\sum_{i=1}^{n} F_i = F_{n+2} - 1, \qquad \sum_{i=1}^{n} F_i^2 = F_n F_{n+1}.$$

- 奇偶项求和：

$$\sum_{i=1}^{n} F_{2i-1} = F_{2n}, \qquad \sum_{i=1}^{n} F_{2i} = F_{2n+1} - 1.$$

- 带权和：

$$\sum_{i=1}^{n} iF_i = nF_{n+2} - F_{n+3} + 2.$$

- Zeckendorf 定理：每个正整数都可唯一表示为若干个互不相邻的 Fibonacci 数之和（从 $F_2 = 1$ 开始），构造时从大到小贪心。

位运算与构造

- 恒等式：

$$a + b = (a \oplus b) + 2(a \,\&\, b) = (a \mid b) + (a \,\&\, b).$$

- $a + b$ 与 $a \oplus b$ 奇偶性相同。
- 已知 $X = a + b$、$Y = a \oplus b$，令 $d = (X - Y)/2$。两数非负整数解存在当且仅当

$$X \geq Y, \qquad X - Y \equiv 0 \pmod 2, \qquad (d \,\&\, Y) = 0.$$

有解时可取 $(a, b) = (d, d + Y)$。

- 若改为三个数满足 $a + b + c = X$、$a \oplus b \oplus c = Y$，只需 $X \geq Y$ 且同奇偶；可取 $(a, b, c) = (d, d, Y)$。
- 连续异或满足四周期：

$$0 \oplus 1 \oplus \cdots \oplus n = \begin{cases} n, & n \bmod 4 = 0, \\ 1, & n \bmod 4 = 1, \\ n + 1, & n \bmod 4 = 2, \\ 0, & n \bmod 4 = 3. \end{cases}$$

- 正整数 n 能拆成恰好 k 个 2 的非负整数次幂之和，当且仅当

$$\operatorname{popcount}(n) \leq k \leq n.$$

其他常用定理与模型

- 麦乐鸡定理：$\gcd(a, b) = 1$ 时，不能写成 $xa + yb$（$x, y \geq 0$）的最大非负整数是 $ab - a - b$；所有更大的整数都能表示。
- 抽屉原理：把 N 个对象放入 m 个盒子，至少一个盒子含有不少于 $\lceil N/m \rceil$ 个对象。
- 容斥原理：

$$\left| \bigcup_{i=1}^{n} S_i \right| = \sum_i |S_i| - \sum_{i<j} |S_i \cap S_j| + \sum_{i<j<k} |S_i \cap S_j \cap S_k| - \cdots.$$

- 弱 Goldbach 定理已经证明：每个大于 5 的奇数都能写成三个奇质数之和。强 Goldbach“每个大于 2 的偶数都是两个质数之和”仍是猜想，除非题目允许或范围可验证，否则不能当定理直接使用。

使用建议

1. 模数为质数且只需小范围组合数时，用 `Comb`；`n` 可能超过模数时用 `Lucas`。
2. 数值达到 `1e18` 时，幂乘统一使用 `mul_mod` 或 `i128`；不要直接写 `a*b%mod`。
3. 需要大量最小质因子、`phi`、`mu` 查询时一次构造 `Sieve`，不要每次试除。
4. 单个大整数分解优先使用 `factor`；数据范围只有 `1e12` 且调用很少时，试除法常数更小。
5. CRT 题目若模数不保证互质，直接用 `excrt`；返回 `-1` 时先判断题目是否允许无解。

6. 所有下标、阶乘、答案中间值按题目范围选择 `int` 或 `ll`，本板子接口默认返回 `ll`。